

数据库系统原理实验—实验十二

数据库并发事务实验

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

课 程 教 师: 赖韩江

起 始 日 期: 2025年12月18日

结 束 日 期: 2025年12月25日

实 验 地 点: B202

目录

1 实验概述 **2**
1.1 实验目的 2
1.2 实验内容 2

2 实验报告 **2**
2.1 创建事务并保证数据一致性 2
2.2 观察锁的获得与释放 3
2.3 事务隔离级别模拟 4
2.3.1 脏读 (Read Uncommitted) 模拟与回滚验证 4
2.3.2 不可重复读 (Read Committed) 验证 4
2.3.3 可重复读 (Repeatable Read) 与锁阻塞验证 5
2.4 实验四：模拟并制造事务死锁 (Deadlock) 7
2.4.1 实验步骤 7
2.4.2 实验代码 7
2.4.3 实验结果与现象分析 8

3 实验总结与问题分析 **8**
3.1 主要收获 8
3.2 困难与解决方法 (Troubleshooting) 8

1　实验概述

1.1　实验目的

熟悉数据库并发事务的相关操作实验。

1.2　实验内容

1. 创建一个事务：在record表中插入一条新记录，同时修改book表中的interview_times为interview_times+1，以保证数据的一致性。（参考实验教材4.1）

2. 编写事务程序，用于更新book表中的SQL Server 2012宝典的信息，观察其事务过程中锁的获得与释放情况，以及锁定资源的类型。（参考4.2）

3. 编写事务程序，对book表进行实验，设置相应的隔离级别，模拟实现读脏数据、不可重复读以及可重复读。（参考4.3）

4. 编写事务程序，对book表进行实验，设计实验制造事务之间的死锁。（参考4.4）

2　实验报告

2.1　创建事务并保证数据一致性

实验目的： 验证事务的原子性（Atomicity）。通过编写一个事务，模拟借书操作：在 record 表中插入一条借阅记录，同时将 book 表中对应书籍的“借阅次数”加1。

实验代码：

```
BEGIN TRANSACTION;
-- 1. 插入借阅记录
INSERT INTO dbo.record (reader_id, book_id, borrow_date, return_date, notes)
VALUES ('r0004', 'b0001', GETDATE(), DATEADD(day, 30, GETDATE()), '一致性测试');
-- 2. 更新书籍借阅次数 (interviews_times)
UPDATE dbo.book
SET interviews_times = interviews_times + 1
WHERE book_id = 'b0001';
COMMIT TRANSACTION;
-- 验证数据
SELECT * FROM dbo.record WHERE book_id = 'b0001';
SELECT book_id, interviews_times FROM dbo.book WHERE book_id = 'b0001';
```

实验结果：

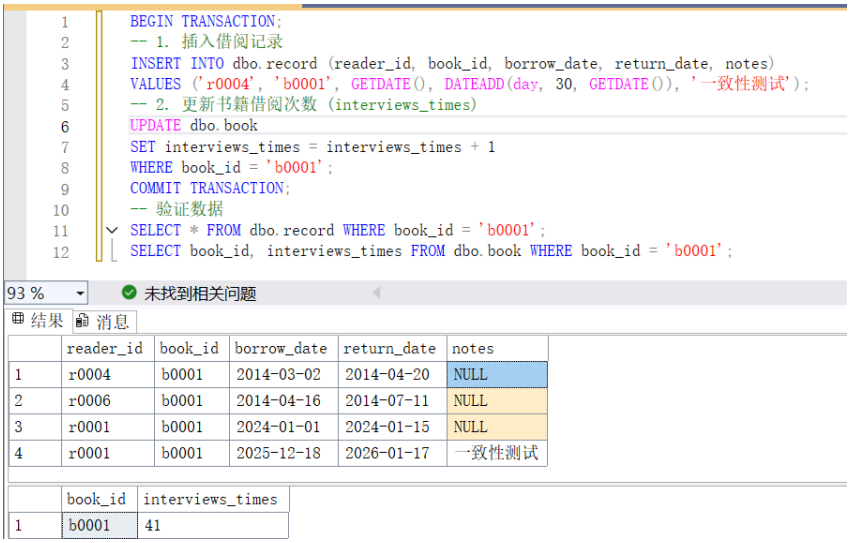


图 1: 数据一致性验证结果

结果分析：如上图所示，查询结果显示 record 表成功插入了新记录，同时 book 表的 interviews_times 字段数值增加了 1。说明事务内的两个操作要么全部成功，保证了数据的一致性。

2.2　观察锁的获得与释放

实验目的： 观察事务在执行过程中对资源锁定的情况。

实验代码：

```
BEGIN TRANSACTION;
-- 更新操作，但在提交前暂停
UPDATE dbo.book SET book_price = 120.00 WHERE book_name = 'SQL Server 宝典2012';

-- 查询当前会话持有的锁
SELECT resource_type, request_mode, resource_description, request_status
FROM sys.dm_tran_locks
WHERE request_session_id = @@SPID;
COMMIT TRANSACTION;
```

实验结果：

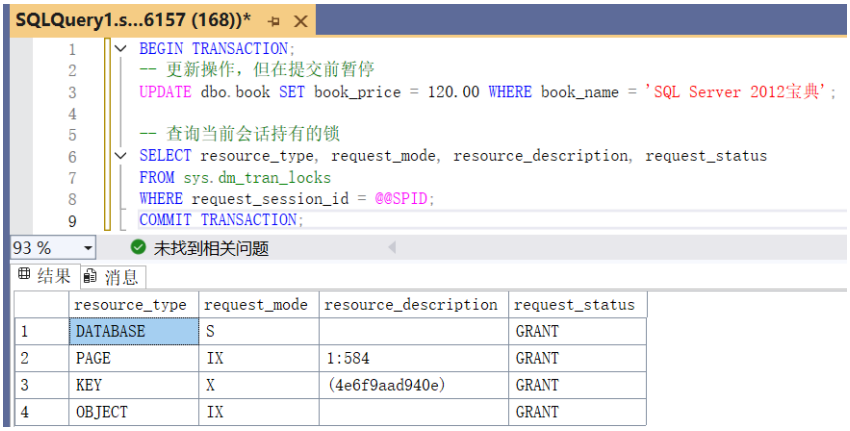


图 2: 锁持有情况查询结果

结果分析：如上图所示，在事务提交之前，系统视图显示当前会话对 book 表的相关资源持有了 X 锁（排他锁），表明在事务结束前，该资源被当前事务独占，其他事务无法修改。

2.3　事务隔离级别模拟

2.3.1　脏读 (Read Uncommitted) 模拟与回滚验证

实验目的： 验证在 READ UNCOMMITTED 隔离级别下，事务可以读取到其他事务未提交的修改。

实验代码：

```
-- 窗口：执行更新但不提交 A
BEGIN TRAN;
UPDATE dbo.book SET interviews_times = 999 WHERE book_id = 'b0001';

-- 窗口：设置隔离级别并查询 B
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SELECT interviews_times FROM dbo.book WHERE book_id = 'b0001';

-- 回滚操作：窗口 A 执行
ROLLBACK;
```

实验结果：

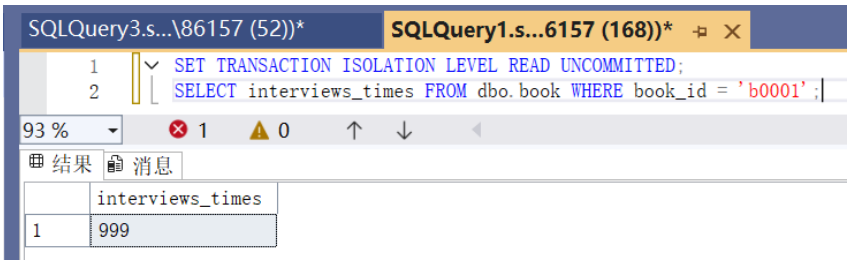


图 3: 窗口 B 读到了窗口 A 尚未提交的脏数据 999

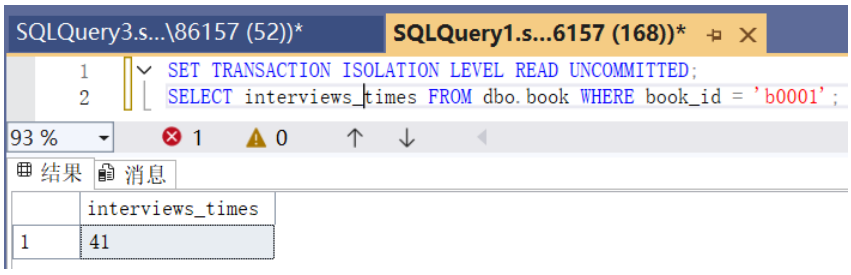


图 4: 窗口 A 回滚后，窗口 B 再次查询发现数据恢复为 41

结果分析： 在 READ UNCOMMITTED 级别下，数据库允许“脏读”。实验显示窗口 B 读取到了窗口 A 未提交的中间值。当窗口 A 回滚后，数据恢复原状，证明了脏读现象的存在。

2.3.2　不可重复读 (Read Committed) 验证

实验目的： 验证在 READ COMMITTED 级别下，同一事务内两次读取之间由于其他事务的提交导致结果不一致。

实验代码：

```
-- 窗口：开启事务并进行第一次读取 A
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
BEGIN TRAN;
SELECT interviews_times FROM dbo.book WHERE book_id = 'b0001';

-- 窗口：修改数据并提交 B
UPDATE dbo.book SET interviews_times = 50 WHERE book_id = 'b0001';
COMMIT;

-- 窗口：再次执行查询 A
SELECT interviews_times FROM dbo.book WHERE book_id = 'b0001';
COMMIT;
```

实验结果：

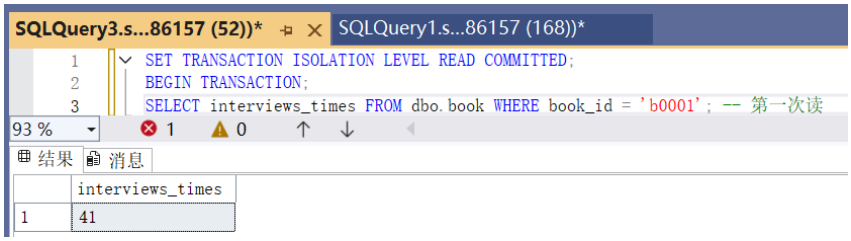


图 5: 窗口 A 第一次读取结果为 41

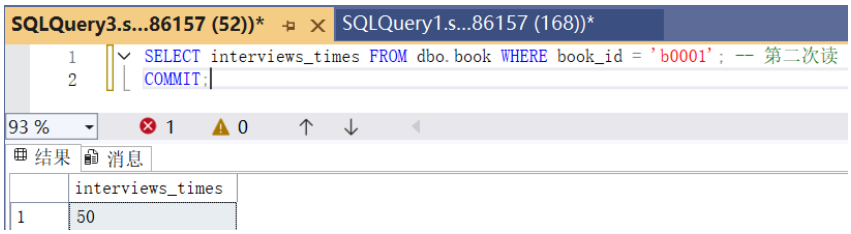


图 6: 窗口 B 提交修改后，窗口 A 第二次读取结果变为了 50

结果分析： READ COMMITTED 级别避免了脏读，但允许不可重复读。窗口 A 在同一事务内的两次读取结果不同，是因为窗口 B 在两次读取之间提交了更新。

2.3.3　可重复读 (Repeatable Read) 与锁阻塞验证

实验目的： 验证 REPEATABLE READ 通过锁定读取的行，确保在事务结束前该行数据不会被其他事务修改。

实验代码：

```
-- 窗口：设置隔离级别并开启事务 A
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN TRAN;
SELECT interviews_times FROM dbo.book WHERE book_id = 'b0001';

-- 窗口：尝试更新同一行数据 B
```

```
UPDATE dbo.book SET interviews_times = 55 WHERE book_id = 'b0001';

-- 窗口：再次查询并提交 A
SELECT interviews_times FROM dbo.book WHERE book_id = 'b0001';
COMMIT;
```

实验结果：

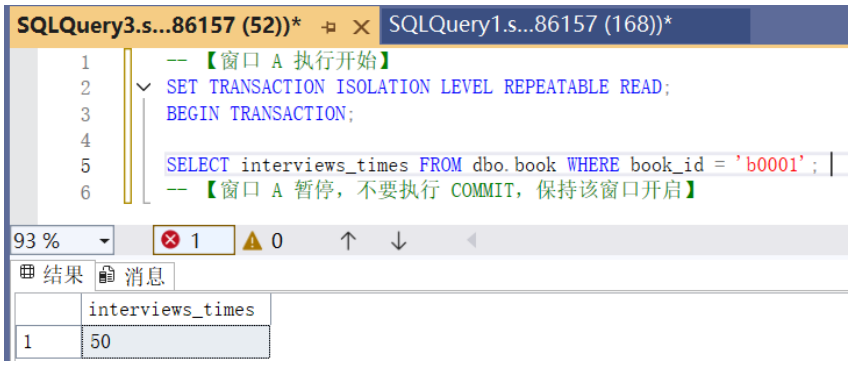


图 7: 窗口 A 启动事务并读取结果为 50

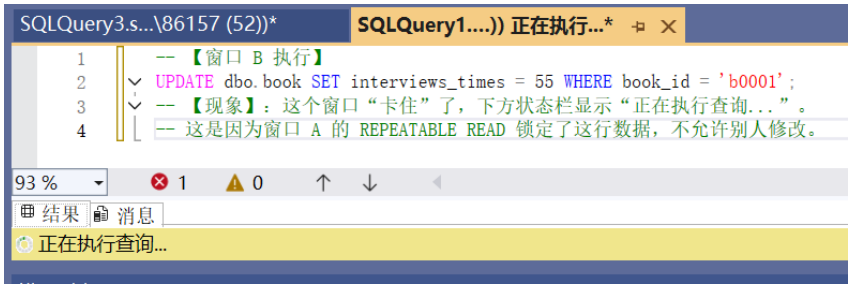


图 8: 窗口 B 执行更新语句时发生阻塞，下方状态栏显示“正在执行查询”，证明窗口 A 持有的 S 锁阻止了窗口 B 获取 X 锁

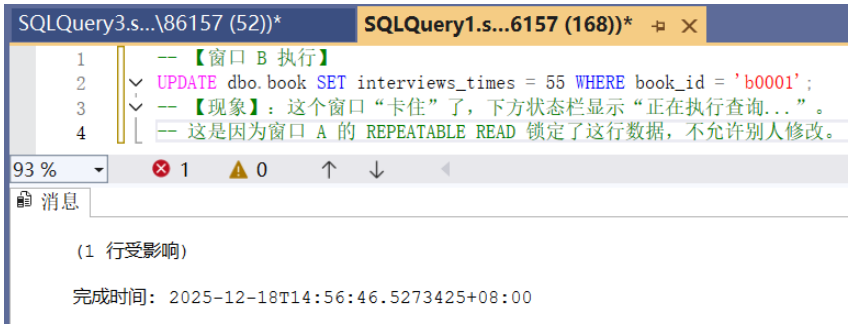


图 9: 窗口 B 在窗口 A commit 后结束阻塞

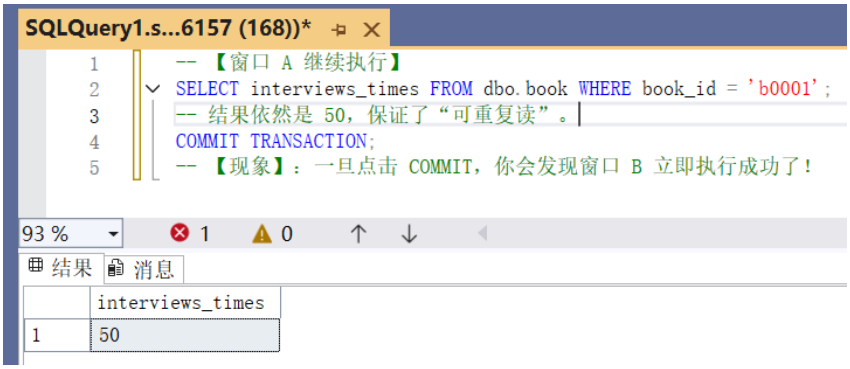


图 10: 窗口 A 再次查询，结果依然是 50，提交事务后窗口 B 才成功执行

结果分析： 在 REPEATABLE READ 级别下，窗口 A 对读取的行持有共享锁（S 锁）直到事务结束。这导致窗口 B 的更新操作（需要排他锁 X 锁）被阻塞。只有当窗口 A 提交事务释放锁后，窗口 B 才能继续执行，从而确保了窗口 A 在事务期间读取数据的一致性。

2.4 实验四：模拟并制造事务死锁 (Deadlock)

2.4.1 实验步骤

1. **准备阶段：** 打开两个 SSMS 查询窗口，均连接至 JY 数据库，并并排显示以便观察时序。
2. **事务构建：** 在两个窗口中分别编写逻辑相反的事务程序。事务 A 先锁书籍 b0001 再请求 b0002；事务 B 先锁 b0002 再请求 b0001。
3. **并发执行：** 几乎同时启动两个窗口的脚本，通过 WAITFOR DELAY 确保两个事务都能先获取到各自的第一把锁，从而形成循环等待。

2.4.2 实验代码

Listing 1: 窗口 1：事务 A

```
-- 事务：先锁 A ，再请求b0001 b0002
USE JY; GO
PRINT 事务'开始A...';
BEGIN TRANSACTION;
  UPDATE book
  SET book_price = book_price + 1
  WHERE book_id = 'b0001';

  PRINT 事务'已锁Ab0001, 等待秒5...';
  WAITFOR DELAY '00:00:05';

  UPDATE book
  SET book_price = book_price + 1
  WHERE book_id = 'b0002';
COMMIT TRANSACTION;
```

Listing 2: 窗口 2：事务 B

```
-- 事务：先锁 B ，再请求b0002 b0001
USE JY; GO
PRINT 事务'开始B...';
BEGIN TRANSACTION;
  UPDATE book
  SET book_price = book_price - 1
  WHERE book_id = 'b0002';

  PRINT 事务'已锁Bb0002, 等待秒5...';
  WAITFOR DELAY '00:00:05';

  UPDATE book
  SET book_price = book_price - 1
  WHERE book_id = 'b0001';
COMMIT TRANSACTION;
```

2.4.3　实验结果与现象分析

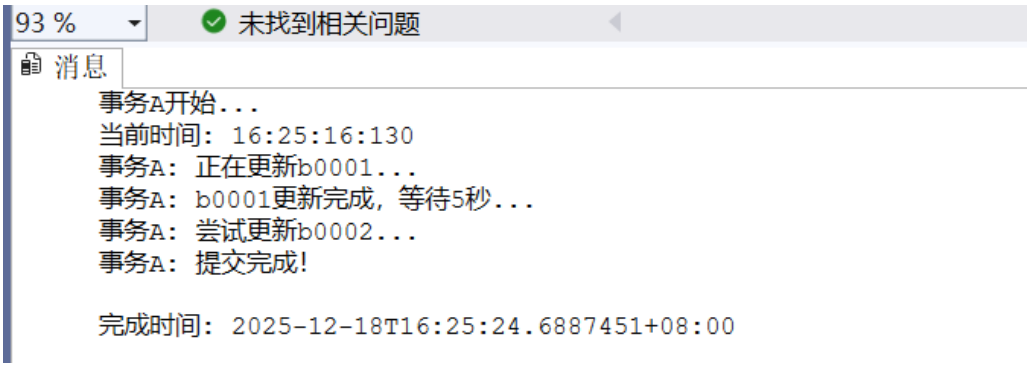


图 11: 事务 A 执行成功：作为死锁竞争中的幸存者正常提交

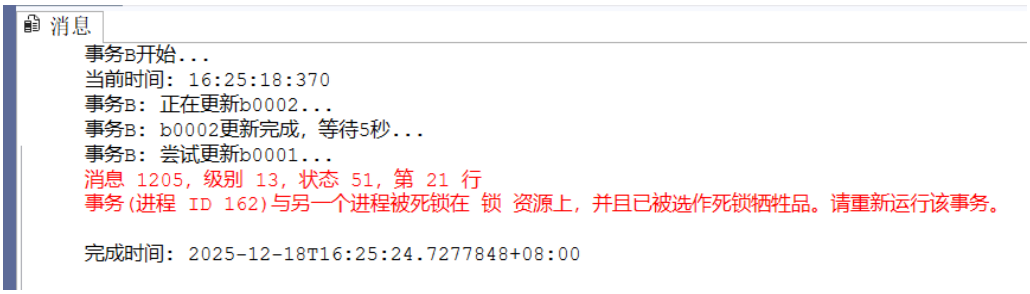


图 12: 事务 B 执行失败：触发 1205 错误，被选作死锁牺牲品

结果判定： 实验成功触发死锁。如图所示，事务 B 在尝试更新 b0001 时，系统检测到循环等待，报错：“*消息 1205, 级别 13, 事务（进程 ID 162）与另一个进程被死锁在锁资源上，并且已被选作死锁牺牲品*”。此时事务 B 自动回滚，释放锁资源，使得事务 A 能够完成后续更新并成功提交。

3　实验总结与问题分析

3.1　主要收获

通过本次实验，我系统地理解了事务的 ACID 特性以及并发控制的重要性。我掌握了如何通过设置 TRANSACTION ISOLATION LEVEL 来控制并发副作用，并学会了利用系统动态管理视图监控锁资源。

3.2　困难与解决方法 (Troubleshooting)

在实验过程中，我遇到了以下代表性的技术问题，并逐一进行了排查与解决：

1. **非预期的事务阻塞 (Blocking)：** 在模拟死锁之前，我发现即使只执行一条 UPDATE 语句，窗口也会长时间处于“正在执行查询”状态（见图 bug.png）。

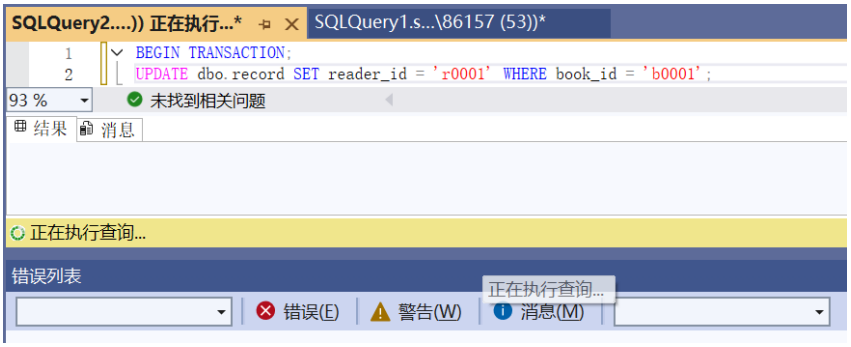
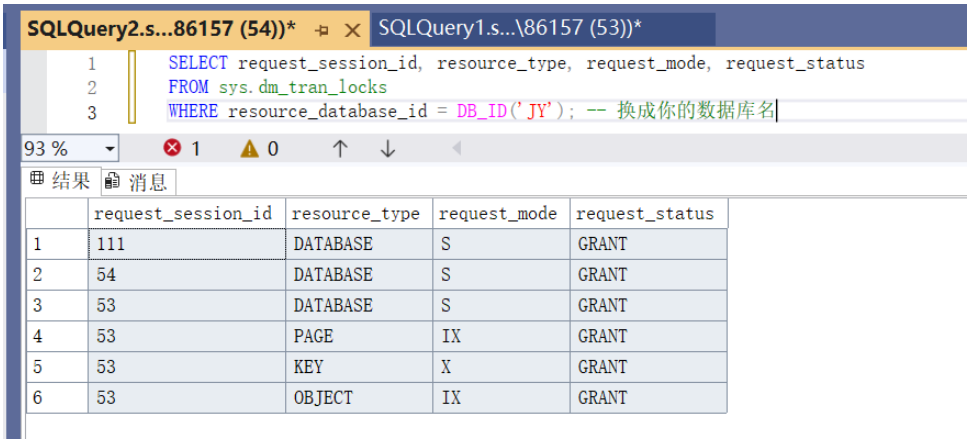


图 13: 非预期阻塞现象：窗口处于持续挂起状态

2. **死锁残留与后台进程监控：** 通过查询 `sys.dm_tran_locks` 视图（见图 `bug2.png`），我发现会话 53 持有了 `KEY` 的排他锁 (`X`)。这说明之前的实验事务未正常回滚或提交，导致资源被永久锁定。



	request_session_id	resource_type	request_mode	request_status
1	111	DATABASE	S	GRANT
2	54	DATABASE	S	GRANT
3	53	DATABASE	S	GRANT
4	53	PAGE	IX	GRANT
5	53	KEY	X	GRANT
6	53	OBJECT	IX	GRANT

图 14: 利用 DMV 视图排查锁资源：Session 53 持有排他锁

3. **约束冲突问题：** 在更新 `record` 表时，曾因修改外键字段导致 `547` 约束冲突，或修改主键字段导致 `2627` 冲突。

最终解决方法： 针对上述问题，我采取了以下措施：首先通过执行 `ROLLBACK` 或 `KILL` 命令强制关闭残留的僵尸进程；其次，在死锁实验中将更新对象统一为非约束字段（如 `book_price`），避开了业务逻辑干扰。这些调试过程加深了我对数据库排他锁 (X Lock) 与意向排他锁 (IX Lock) 互斥关系的理解。